EzPay 백엔드 개발 포트폴리오

> 간편 송금 서비스 · 백엔드 개발자 관점

목차

1. 표지용 프로젝트 소개
2. 프로젝트 개요
3. 담당 역할 (백엔드 개발자 관점)
4. 시스템 아키텍처
5. 핵심 기술 선택 이유
6. 핵심 구현 내용
7. 트러블슈팅 / 기술적 문제 해결 사례
8. 성능 · 안정성 · 운영 관점의 강점
9. 협업 관점에서의 어필 포인트
10. 회고 및 개선 방향
11. 한 페이지 요약본
12. 면접 예상 질문과 답변 포인트
13. PDF 디자인 반영 시 추천 레이아웃

1. 표지용 프로젝트 소개

EzPay는 사용자가 계좌를 만들고 송금하며 거래 내역과 통계를 확인할 수 있는 간편 송금 서비스입니다. 단순한 CRUD 애플리케이션이 아니라, 돈이 오가는 금융 서비스의 백엔드가 갖춰야 할 인증·인가 보안, 비동기 이벤트 처리, 실시간 관리자 모니터링, 관측 가능성(Observability)을 모두 고려해 설계했습니다. Java 17과 Spring Boot 3.4 기반 위에 Spring Security와 JWT로 인증 체계를 세우고, Kafka로 거래 이벤트를 비동기로 처리하며, WebSocket·SSE로 관리자 실시간 대시보드를 지원하고, Prometheus·Grafana·Sentry로 운영 중 시스템 상태를 추적할 수 있도록 구성했습니다. 도메인은 account, payment, auth, notification, realtime, risk, analytics, admin 등으로 모듈화해 유지보수성과 확장성을 확보했습니다.

2. 프로젝트 개요

핵심 요약 EzPay는 회원가입/로그인, 계좌 관리, 송금, 거래 내역 및 통계, 관리자 대시보드를 제공하는 간편 송금 백엔드 서비스입니다. 금융 도메인 특성상 보안과 데이터 정합성을 최우선으로 두고, 대량 거래 상황에서도 안정적으로 동작하도록 비동기·실시간 처리 구조를 마련했습니다.

상세 설명 서비스는 크게 세 축으로 구성됩니다. 첫째는 일반 사용자 기능으로, 회원가입과 JWT 기반 로그인, 계좌 생성 및 관리, 송금, 거래 내역 조회, 계좌 통계, 비밀번호 재설정, 알림을 제공합니다. 둘째는 이벤트 처리 축으로, 송

금과 같은 핵심 도메인 이벤트를 Kafka로 발행·소비해 알림·통계·리스크 판단 등 후속 처리를 본 거래 흐름과 분리했습니다. 셋째는 운영·관리 축으로, 관리자 대시보드와 WebSocket/SSE 기반 실시간 모니터링, Actuator·Prometheus 메트릭, Sentry 오류 추적을 통해 서비스 상태를 실시간으로 관찰할 수 있게 했습니다.

도메인은 account, admin, analytics, auth, notification, payment, realtime, risk, systemlog, user 등 책임 단위로 나누고, 그 아래 shared·security·config·common 레이어를 두어 공통 관심사(보안 설정, 예외 처리, 공용 유틸)를 도메인과 분리했습니다. 배포는 Render, 데이터베이스는 AWS RDS(PostgreSQL)를 사용하며, 운영/로컬 프로파일과 환경변수 기반 설정으로 환경별 구성을 분리했습니다.

사용 기술 Java 17, Spring Boot 3.4.2, Spring Security 6.4.5, JPA/Hibernate, QueryDSL, PostgreSQL, JWT, Kafka, WebSocket, SSE, Actuator, Micrometer Prometheus, Sentry, Gradle / AWS RDS, Render, Prometheus, Grafana

성과 · 배운 점 금융 서비스 백엔드에서 요구되는 보안·정합성·실시간성·운영성을 하나의 프로젝트 안에서 종합적으로 설계하고 검증했습니다. 특히 "핵심 거래 흐름"과 "부가 처리 흐름"을 분리하는 것이 안정성의 출발점이라는 점을 구조 설계 과정에서 체득했습니다.

---

3. 담당 역할 (백엔드 개발자 관점)

핵심 요약 백엔드 개발자로서 인증/인가 체계, 송금 및 계좌 도메인, Kafka 기반 이벤트 파이프라인, 실시간 모니터링, 관측 가능성 구성 등 서버 사이드 전반을 담당했습니다.

상세 설명 담당한 범위는 다음과 같습니다.

- **인증/인가 설계**: Spring Security 6와 JWT를 결합한 인증 필터 체인, 토큰 발급/검증 로직, 권한 기반 접근 제어, 내부 API 시크릿 키 관리를 설계·구현했습니다.
- **핵심 도메인 개발**: 계좌 생성·관리, 송금, 거래 내역 조회, 계좌 통계 등 payment·account 도메인의 비즈니스 로직과 QueryDSL 기반 동적 조회를 담당했습니다.
- **이벤트 처리 파이프라인**: 송금 등 도메인 이벤트를 Kafka로 발행하고, consumer와 DLT(Dead Letter Topic) 처리로 실패 이벤트를 격리하는 비동기 처리 구조를 구현했습니다.
- **실시간 기능**: 관리자용 WebSocket/SSE 스트리밍과 실시간 파이프라인 스케줄러를 구현해 운영자가 시스템 상태를 실시간으로 볼 수 있게 했습니다.
- **관측 가능성 및 운영**: Actuator, Micrometer Prometheus 메트릭 노출, Grafana 대시보드 연동, Sentry 오류 추적을 구성하고, 운영/로컬 프로파일과 CORS·환경변수 설정을 관리했습니다.

사용 기술 Spring Security, JWT, JPA/Hibernate, QueryDSL, Kafka, WebSocket, SSE, Actuator, Micrometer, Sentry

성과 · 배운 점 기능 구현에 그치지 않고 "이 코드가 운영 중 장애가 나면 어떻게 감지하고 격리할 것인가"까지 책임 범위로 인식하게 됐습니다. 인증, 도메인, 비동기, 실시간, 관측을 하나로 잇는 경험을 통해 백엔드를 시스템 관점에서 바라보는 시야를 얻었습니다.

---

4. 시스템 아키텍처

핵심 요약 클라이언트 요청은 인증 필터를 통과한 뒤 도메인 서비스에서 처리되고, 부가 처리는 Kafka 이벤트로 분리되며, 운영자는 실시간 스트림과 모니터링 스택으로 시스템을 관찰합니다.

상세 설명 요청 흐름은 다음 단계로 구성됩니다.

1. **진입 및 인증**: 클라이언트 요청은 CORS 정책과 JWT 인증 필터를 통과합니다. 유효한 토큰이면 SecurityContext에 인증 정보가 채워지고, 권한에 따라 접근이 제어됩니다.
2. **도메인 처리**: auth, account, payment 등 도메인 서비스가 요청을 처리합니다. 조회 계열은 QueryDSL로 동적 쿼리를 구성하고, 데이터는 PostgreSQL(AWS RDS)에 저장됩니다.
3. **비동기 이벤트**: 송금 성공 같은 핵심 이벤트는 Kafka 토픽으로 발행됩니다. notification, analytics, risk 도메인의 consumer가 이를 구독해 알림 발송, 통계 집계, 리스크 판단 등 후속 작업을 본 트랜잭션과 분리해 처리합니다. 처리 실패 이벤트는 DLT로 격리해 재처리·분석이 가능하도록 했습니다.
4. **실시간 스트리밍**: realtime 도메인은 스케줄러 기반 파이프라인으로 데이터를 가공하고, WebSocket/SSE를 통해 관리자 대시보드로 실시간 스트림을 내보냅니다.
5. **관측 및 운영**: Actuator와 Micrometer가 애플리케이션 메트릭을 노출하고 Prometheus가 수집, Grafana가 시각화합니다. 예외는 Sentry로 수집되어 알림과 추적이 이뤄집니다.

레이어 구조는 도메인 모듈(account, admin, analytics, auth, notification, payment, realtime, risk, systemlog, user)과 횡단 관심사 레이어(shared, security, config, common)로 나뉩니다. 이 분리 덕분에 보안 설정이나 공통 예외 처리를 한 곳에서 관리하면서 도메인 로직은 각자의 책임에 집중할 수 있습니다.

사용 기술 Spring Boot, Spring Security, JWT, Kafka, WebSocket, SSE, QueryDSL, PostgreSQL(AWS RDS), Prometheus, Grafana, Sentry, Render

성과 · 배운 점 "동기로 처리할 것과 비동기로 미룰 것", "실시간으로 밀어줄 것과 배치로 집계할 것"을 구분하는 아키텍처 감각을 길렀습니다. 핵심 거래 경로를 얇고 빠르게 유지하고 나머지를 이벤트로 분리하는 설계가 안정성과 확장성의 핵심임을 확인했습니다.

---

5. 핵심 기술 선택 이유

핵심 요약 각 기술은 금융 서비스의 요구사항(보안, 정합성, 비동기, 실시간, 관측)에 대응하기 위해 선택했으며, 대안 대비 어떤 판단으로 채택했는지를 기준으로 정리했습니다.

상세 설명

- **Spring Security 6 + JWT**: 세션 기반 인증 대신 무상태(stateless) JWT를 택해 서버 확장성과 수평 확장에 유리하도록 했습니다. 토큰에 최소 정보만 담고 서버에서 검증하는 방식으로, 인증 상태를 서버 메모리에 두지 않아 배포·확장 시 부담을 줄였습니다.
- **Kafka (vs 동기 호출/단순 큐)**: 송금 이후의 알림·통계·리스크 처리를 동기로 붙이면 하나의 실패가 거래 전체를 지연시키거나 실패시킬 수 있습니다. Kafka로 이벤트를 분리하고 DLT로 실패를 격리해, 부가 기능 장애가 핵심 거래에 전파되지 않도록 했습니다.
- **QueryDSL (vs 문자열 JPQL)**: 거래 내역·통계처럼 조건이 동적으로 바뀌는 조회에서 타입 안전성과 컴파일 타임 검증을 확보하기 위해 채택했습니다. 조건 조합이 많은 화면일수록 유지보수성이 크게 향상됩니다.
- **WebSocket/SSE**: 관리자 대시보드는 서버가 능동적으로 상태를 밀어주는 것이 적합해, 폴링 대신 서버 푸시 방식을 택했습니다. 단방향 스트림 성격에는 SSE를, 상호작용이 필요한 채널에는 WebSocket을 활용했습니다.
- **Prometheus + Grafana + Sentry**: 메트릭(Prometheus/Grafana)과 오류 추적(Sentry)을 분리해, 시스템 지표 추세와 개별 예외 상세를 각각의 도구로 관측하도록 했습니다.
- **PostgreSQL (AWS RDS)**: 트랜잭션 정합성과 안정적 운영이 중요한 금융 데이터에 적합하고, RDS로 운영 부담을 낮췄습니다.

사용 기술 위 전 항목

성과 · 배운 점 "왜 이 기술인가"를 대안과의 트레이드오프로 설명할 수 있게 됐습니다. 기술 선택을 유행이 아니라 요구사항과 실패 시나리오 기준으로 판단하는 습관을 얻었습니다.

---

6. 핵심 구현 내용

6-1. JWT 기반 인증/인가 체계

핵심 요약 Spring Security 6 필터 체인에 JWT 검증 필터를 통합해 무상태 인증을 구현하고, 권한 기반 접근 제어와 내부 API 시크릿 키 관리를 함께 설계했습니다.

상세 설명 로그인 시 토큰을 발급하고, 이후 요청마다 커스텀 필터가 토큰을 검증해 SecurityContext를 채웁니다. 공개 엔드포인트, 인증 필요 엔드포인트, 관리자 전용 엔드포인트를 구분해 접근을 제어하고, 서비스 간 내부 호출에는 별도의 시크릿 키를 두어 외부 사용자 인증과 내부 신뢰 경계를 분리했습니다. 비밀번호 재설정 같은 민감 플로우도 별도로 다뤘습니다.

사용 기술 Spring Security 6, JWT, 환경변수 기반 시크릿 관리

성과 · 배운 점 인증(Authentication)과 인가(Authorization), 그리고 내부/외부 신뢰 경계를 명확히 나누는 보안 설계 관점을 확립했습니다.

6-2. 송금 및 계좌 도메인

핵심 요약 계좌 생성·관리와 송금, 거래 내역, 계좌 통계를 정합성 있게 처리하도록 도메인 로직을 구현했습니다.

상세 설명 송금은 출금과 입금이 하나의 일관된 결과로 반영되어야 하는 만큼 트랜잭션 경계를 신중히 설계했습니다. 거래 내역과 계좌 통계 조회는 조건이 다양해 QueryDSL로 동적 쿼리를 구성했고, 성공한 송금 이벤트는 후속 처리를 위해 Kafka로 발행되도록 연결했습니다.

사용 기술 JPA/Hibernate, QueryDSL, PostgreSQL, Kafka

성과 · 배운 점 금융 도메인에서 데이터 정합성과 트랜잭션 경계 설정의 중요성을 체감했습니다.

6-3. Kafka 기반 비동기 이벤트 처리와 DLT

핵심 요약 핵심 거래 이후의 부가 처리를 Kafka 이벤트로 분리하고, 실패 이벤트를 DLT로 격리해 장애가 전파되지 않도록 했습니다.

상세 설명 송금 성공 이벤트를 발행하면 notification, analytics, risk 등의 consumer가 각자 구독해 알림, 통계, 리스크 판단을 수행합니다. consumer 처리 중 오류가 발생하면 무한 재시도로 파이프라인을 막지 않고 DLT로 이동시켜, 문제 이벤트를 따로 재처리·분석할 수 있게 했습니다.

사용 기술 Kafka(Producer/Consumer, DLT)

성과 · 배운 점 "실패를 어디로 보낼 것인가"를 미리 설계하는 것이 비동기 시스템 안정성의 핵심임을 배웠습니다.

6-4. 관리자 실시간 대시보드 (WebSocket/SSE)

핵심 요약 관리자가 시스템·거래 상태를 실시간으로 볼 수 있도록 스케줄러 기반 실시간 파이프라인과 WebSocket/SSE 스트리밍을 구현했습니다.

상세 설명 realtime 도메인의 스케줄러가 주기적으로 데이터를 가공해 스트림으로 내보내고, 관리자 대시보드는 이를 실시간으로 수신합니다. 폴링 방식 대비 서버 푸시로 지연을 줄이고 불필요한 요청을 없앴습니다.

사용 기술 WebSocket, SSE, 스케줄러

성과 · 배운 점 실시간 요구사항에서 채널 특성(단방향/양방향)에 맞는 프로토콜 선택의 중요성을 익혔습니다.

6-5. 관측 가능성 (Metrics · Tracing · Error)

핵심 요약 Actuator·Micrometer로 메트릭을 노출하고 Prometheus/Grafana로 시각화, Sentry로 오류를 추적하는 관측 스택을 구성했습니다.

상세 설명 애플리케이션 상태를 지표로 드러내 Grafana 대시보드에서 추세를 확인하고, 예외는 Sentry로 모아 알림받도록 했습니다. 운영 중 문제를 코드 배포 없이도 관찰·진단할 수 있는 기반을 마련했습니다.

사용 기술 Actuator, Micrometer Prometheus, Prometheus, Grafana, Sentry

성과 · 배운 점 "운영은 관찰에서 시작한다"는 관점으로 개발 단계부터 관측 가능성을 설계에 포함하게 됐습니다.

6-6. 모듈 분리와 보안·설정 레이어

핵심 요약 도메인 모듈과 shared/security/config/common 레이어를 분리해 유지보수성과 보안 관리 일관성을 확보했습니다.

상세 설명 도메인별 책임을 나누고, 보안 설정·CORS·공통 예외·공용 유틸을 횡단 레이어에 모아 한 곳에서 관리했습니다. 운영/로컬 프로파일과 환경변수 기반 설정으로 환경별 구성을 안전하게 분리했습니다.

사용 기술 Spring 프로파일, 환경변수 설정, CORS 구성

성과 · 배운 점 관심사 분리가 기능 추가·변경 시 영향 범위를 좁혀 유지보수 비용을 낮춘다는 점을 확인했습니다.

---

7. 트러블슈팅 / 기술적 문제 해결 사례

사례 1. 부가 처리 실패가 핵심 거래에 전파되던 문제

문제 초기 설계에서는 송금 이후 알림·통계 처리를 동기적으로 이어 붙였습니다. 이 경우 후속 처리에서 오류가 나면 사용자 입장의 거래 흐름까지 영향을 받을 위험이 있었습니다.

원인 핵심 거래와 부가 처리가 하나의 실행 흐름에 묶여 있어 실패가 격리되지 않았습니다.

해결 송금 성공 이벤트를 Kafka로 발행하고, 부가 처리를 별도 consumer로 분리했습니다. consumer 실패 이벤트는 DLT로 격리해 재처리·분석이 가능하게 했습니다.

결과 부가 기능 장애가 핵심 거래로 전파되지 않는 장애 격리 구조를 확보하고, 운영 안정성을 강화했습니다.

사례 2. 관리자 실시간 데이터의 지연과 과도한 폴링

문제 관리자 대시보드가 상태를 최신으로 유지하기 위해 반복 조회(폴링)에 의존하면 지연과 불필요한 부하가 발생할 수 있었습니다.

원인 클라이언트가 주기적으로 요청하는 방식은 실시간성과 효율 모두에서 한계가 있었습니다.

해결 realtime 스케줄러로 데이터를 가공해 WebSocket/SSE로 서버가 능동적으로 밀어주는 구조로 전환했습니다. 채널 성격에 맞춰 단방향 스트림에는 SSE를 사용했습니다.

결과 폴링 대비 지연을 줄이고 불필요한 요청을 제거해 실시간 처리 기반을 마련했습니다.

사례 3. 운영 중 문제를 사후에야 인지하던 관측 공백

문제 초기에는 운영 중 오류나 성능 이상을 로그를 뒤져 사후에 파악해야 했습니다.

원인 시스템 상태를 실시간으로 드러내는 메트릭·오류 추적 체계가 부족했습니다.

해결 Actuator·Micrometer로 메트릭을 노출하고 Prometheus/Grafana로 시각화, Sentry로 예외를 수집·알림하도록 관측 스택을 구성했습니다.

결과 문제를 조기에 감지하고 원인 추적 시간을 단축할 수 있는 기반을 만들어 운영 안정성을 강화했습니다.

---

8. 성능 · 안정성 · 운영 관점의 강점

핵심 요약 핵심 거래 경로를 얇게 유지하고 부가 처리를 비동기로 분리했으며, 관측 가능성과 환경 분리로 운영 안정성을 확보했습니다.

상세 설명 - **장애 격리**: Kafka + DLT로 부가 처리 실패가 핵심 거래에 전파되지 않도록 격리했습니다. - **실시간성**: WebSocket/SSE 서버 푸시로 관리자 모니터링의 지연을 줄였습니다. - **관측 가능성**: 메트릭(Prometheus/Grafana)과 오류 추적(Sentry)을 분리 구성해 추세와 상세를 함께 관찰합니다. - **보안**: 무상태 JWT 인증, 권한 기반 접근 제어, 내부 API 시크릿 분리, CORS 관리로 신뢰 경계를 명확히 했습니다. - **운영성**: 운영/로컬 프로파일과 환경변수 기반 설정, AWS RDS·Render 배포로 환경별 구성을 안전하게 분리했습니다. - **조회 효율**: QueryDSL 동적 쿼리로 다양한 조건의 거래·통계 조회를 타입 안전하게 처리했습니다.

사용 기술 Kafka/DLT, WebSocket/SSE, Prometheus/Grafana/Sentry, Spring Security/JWT, QueryDSL

성과 · 배운 점 "안정성은 기능이 아니라 구조에서 나온다"는 관점을 갖게 됐습니다.

---

9. 협업 관점에서의 어필 포인트

핵심 요약 모듈 분리와 관측 가능성, 명확한 신뢰 경계는 다인 협업과 운영 인수인계에 유리한 구조입니다.

상세 설명 도메인별 모듈 분리와 횡단 레이어(shared/security/config/common) 구조는 여러 명이 서로 다른 도메인을 동시에 작업해도 충돌을 줄여줍니다. 관측 스택(Grafana 대시보드, Sentry 알림)은 특정 개인의 암묵지에 의존하지 않고 팀 전체가 시스템 상태를 공유하게 합니다. 운영/로컬 프로파일과 환경변수 기반 설정은 새 팀원의 로컬 환경 구축과 배포 환경 인수인계를 단순하게 만듭니다. 기술 선택 시 대안과 트레이드오프를 기준으로 판단한 경험은, 팀 내 기술 논의에서 "왜"를 설명하는 커뮤니케이션에도 그대로 적용됩니다.

사용 기술 모듈화 구조, Grafana/Sentry 공유, 프로파일·환경변수 설정

성과 · 배운 점 구조와 관측 가능성이 곧 협업 비용을 낮추는 자산이라는 점을 확인했습니다.

---

10. 회고 및 개선 방향

핵심 요약 금융 백엔드로서 안정성·보안·실시간성·운영성을 종합적으로 설계한 경험이 가장 큰 소득이며, 정량 지표 확보와 테스트·부하 검증을 다음 과제로 봅니다.

상세 설명 잘한 점은 핵심 거래 경로와 부가 처리를 분리하고, 관측 가능성을 처음부터 설계에 포함한 것입니다. 아쉬운 점과 개선 방향은 다음과 같습니다. 첫째, 성능·안정성 개선을 정성적으로만 서술하고 있어 부하 테스트와 지표 기반의 정량 근거를 확보할 필요가 있습니다. 둘째, 자동화 테스트(단위·통합) 커버리지를 넓혀 회귀 안전망을 강화하고자 합니다. 셋째, Kafka consumer의 재처리 정책과 idempotency(멱등성) 보장을 더 정교하게 다듬을 여지가 있습니다. 넷째, 인증 체계에 토큰 재발급·만료 전략과 감사 로그(audit log)를 보강하면 금융 서비스로서 완성도를 더 높일 수 있습니다.

사용 기술 (개선 대상) 부하 테스트 도구, 테스트 프레임워크, Kafka 멱등성/재처리 전략

성과 · 배운 점 완성된 구조 위에 "측정과 검증"을 더하는 것이 다음 단계임을 명확히 인식했습니다.

---

11. 한 페이지 요약본

EzPay — 간편 송금 서비스 백엔드

한 줄 소개: 금융 서비스에 요구되는 보안·정합성·실시간성·운영성을 종합 설계한 간편 송금 백엔드.

- **스택**: Java 17, Spring Boot 3.4.2, Spring Security 6, JPA/QueryDSL, PostgreSQL, JWT, Kafka, WebSocket/SSE, Actuator/Micrometer, Prometheus/Grafana, Sentry / AWS RDS, Render
- **인증**: 무상태 JWT + Spring Security 필터 체인, 권한 기반 접근 제어, 내부 API 시크릿 분리
- **비동기**: 송금 이벤트를 Kafka로 발행, consumer로 알림·통계·리스크 분리, DLT로 실패 격리 → 장애 전파 차단
- **실시간**: realtime 스케줄러 + WebSocket/SSE로 관리자 대시보드 실시간 스트리밍
- **관측**: Actuator/Micrometer 메트릭 → Prometheus/Grafana 시각화, Sentry 오류 추적
- **구조**: account·payment·auth·notification·realtime·risk·analytics·admin 도메인 분리 + shared/security/config/common 횡단 레이어
- **핵심 성과(정성)**: 장애 격리로 운영 안정성 강화, 서버 푸시로 실시간 처리 기반 마련, 관측 스택으로 문제 조기 감지

---

12. 면접 예상 질문과 답변 포인트

Q1. 왜 세션이 아니라 JWT를 선택했나요? 포인트: 무상태 인증으로 수평 확장과 배포에 유리, 서버 메모리에 인증 상태를 두지 않음. 트레이드오프(토큰 탈취·만료·재발급)를 인지하고 내부/외부 신뢰 경계 분리와 시크릿 관리로 보완. 개선 과제로 토큰 재발급·감사 로그 언급.

Q2. 송금 처리에서 Kafka를 왜 도입했고, 실패는 어떻게 다뤘나요? 포인트: 알림·통계·리스크 같은 부가 처리를 동기로 붙이면 하나의 실패가 거래에 전파됨. 이벤트로 분리해 핵심 거래 경로를 얇게 유지하고, 실패 이벤트는 DLT로 격리해 재처리·분석. "실패를 어디로 보낼지 먼저 설계"가 핵심.

Q3. 관리자 실시간 대시보드는 왜 WebSocket/SSE인가요? 폴링과 비교하면? 포인트: 폴링은 지연과 불필요한 부하가 큼. 서버가 능동적으로 밀어주는 푸시가 실시간 모니터링에 적합. 단방향 스트림은 SSE, 양방향 상호작용은 WebSocket으로 채널 특성에 맞춰 선택.

Q4. 운영 중 장애를 어떻게 감지하고 진단하나요? 포인트: Actuator/Micrometer로 메트릭 노출 → Prometheus 수집 → Grafana 시각화로 추세 관찰, Sentry로 예외 수집·알림. 지표(추세)와 오류(상세)를 분리 관측해 조기 감지·원인 추적 시간 단축.

Q5. 도메인/레이어를 왜 그렇게 나눴고, 협업에 어떤 이점이 있나요? 포인트: account·payment·auth 등 책임 단위 도메인 분리 + shared/security/config/common 횡단 레이어로 공통 관심사 일원화. 영향 범위를 좁혀 유지보수·병렬 작업·인수인계 비용을 낮춤.

---

13. PDF 디자인 반영 시 추천 레이아웃

전체 톤 & 그리드 A4 세로, 좌우 여백 20mm. 본문 10.5~11pt, 제목 계층은 H1(프로젝트/섹션 대제목)–H2(하위 섹션)–본문 3단계로 단순화. 색은 무채색 베이스(딥네이비/차콜)에 포인트 컬러 1개(예: 청록·블루)만 사용해 금융 서비스의 신뢰감을 주고 가독성을 유지. 상단·하단에 얇은 러닝헤더/푸터(프로젝트명 + 페이지 번호).

페이지 배분(3~5p 기준) - **1p 표지 + 개요**: 상단에 프로젝트명(EzPay)·부제(간편 송금 서비스 백엔드), 표지용 소개 1문단, 하단에 핵심 스택을 배지(태그) 형태로 나열. "한 줄 카피"를 표지 헤드라인으로 크게. - **2p 아키텍처 + 기술 선택 이유**: 상단 절반은 아키텍처 다이어그램(요청→인증→도메인→Kafka→consumer→실시간/관측)을 좌→우 흐름의 박스+화살표로. 하단 절반은 기술 선택 이유를 "기술 / 선택 이유 / 대안 대비" 3열 표로. - **3p 핵심 구현 내용**: 6개 구현을 2열 카드 그리드로. 각 카드는 아이콘 + 소제목 + 2~3줄 요약 + 하단 스택 태그. 밀도가 높으니 여백을 넉넉히. - **4p 트러블슈팅 + 강점**: 트러블슈팅 3건을 "문제 → 원인 → 해결 → 결과" 4단 세로 타임라인 또는 좌측 라벨 표로. 하단에 성능·안정성·운영 강점을 아이콘 리스트로. - **5p 요약 + 면접 포인트 + 회고**: 한 페이지 요약본을 상단 하이라이트 박스로, 면접 강조 포인트 5개를 번호 배지 리스트로, 회고를 마무리 문단으로.

시각 요소 가이드 아키텍처·이벤트 흐름은 반드시 다이어그램으로(글보다 흐름이 빨리 읽힘). 스택은 텍스트 나열 대신 태그/배지. 트러블슈팅은 "문제→해결→결과"가 한눈에 들어오도록 컬러 라벨로 단계 구분. 표는 얇은 구분선과 헤더 음영만 사용해 과하지 않게. 정성적 성과 문구(운영 안정성 강화 등)는 별색 강조로 시선을 유도.

부록 A. 포트폴리오 PDF 편집용 한 줄 카피

> "송금 그 자체보다, 송금이 실패해도 무너지지 않는 구조를 설계했습니다 — 보안·비동기·실시간·관측을 갖춘 금융 백엔드."

대안 카피: - "단순 CRUD가 아니라, 장애를 격리하고 실시간으로 관찰 가능한 간편 송금 백엔드." - "Spring Security·Kafka·SSE·Prometheus로 안정성과 운영성을 함께 설계한 EzPay 백엔드."

부록 B. 면접에서 강조할 포인트 5개

1. **장애 격리 설계**: Kafka + DLT로 부가 처리 실패가 핵심 거래에 전파되지 않게 한 점.
2. **보안 경계 설계**: 무상태 JWT, 권한 기반 접근 제어, 내부 API 시크릿 분리로 신뢰 경계를 명확히 한 점.
3. **실시간 모니터링**: WebSocket/SSE 서버 푸시로 관리자 대시보드 실시간성을 확보한 점.
4. **관측 가능성**: 메트릭(Prometheus/Grafana)과 오류 추적(Sentry)을 분리 구성해 운영 문제를 조기에 감지하도록 한 점.
5. **구조적 유지보수성**: 도메인 모듈 + 횡단 레이어 분리로 협업·확장·인수인계 비용을 낮춘 점.

부록 C. 추가로 보강하면 좋은 수치 / 근거

- **부하 테스트 지표**: 초당 처리 송금 건수(TPS), p95/p99 응답 지연, 동시 접속 처리량 (예: k6, JMeter, Gatling 결과).
- **비동기 효과 비교**: 동기 처리 대비 핵심 거래 응답 시간 단축폭, Kafka consumer 처리 지연(lag) 추이.
- **DLT/재처리 통계**: 실패 이벤트 비율, DLT 재처리 성공률, 평균 재처리 시간.
- **관측 효과**: 장애 감지 시간(MTTD)·복구 시간(MTTR) 개선폭, Sentry 오류 발생 건수 추이.

- **실시간 스트림 성능**: WebSocket/SSE 동시 연결 수, 스트림 전송 지연, 폴링 대비 요청 수 감소율.
- **인증 관련**: 토큰 검증 평균 소요 시간, 인증 실패/차단 시도 로그 통계.
- **테스트 커버리지**: 단위·통합 테스트 커버리지(%)와 핵심 도메인 커버 여부.
- **DB 조회 성능**: QueryDSL 적용 전후 주요 통계 쿼리 응답 시간 및 실행 계획 개선.